

Ops-Ready Runbook

Agentic PR Review & Release Notes Pipeline — Operations Guide

System Snapshot

Component	What it does	Where
orchestrator.py	Wires planner -> reviewer -> release-manager, enforces reliability controls	repo root
mcp/storage_server.py	Persistent memory (review history, calibration log)	mcp/
mcp/retrieval_server.py	TF-IDF retrieval over indexed PR diffs	mcp/
tests/test_policy.py	CI-enforced governance policy checks	tests/
.github/workflows/policy-checks.yml	Runs policy, eval, conversion, and retrieval checks on relevant PRs	.github/workflows/

1. What to Monitor

Signal	Where to check	Healthy baseline
Policy test pass rate	CI run of tests/test_policy.py	5/5 passing (see tool-evolution-drill.md for what a failure looks like)
Retrieval hit rate	mcp/retrieval_server.py --selftest	4/6 or better on the ground-truth query set (docs/retrieval-quality-report.md)
Token budget usage	orchestrator.py output: budget.tokens_used	Well under PER_WORKFLOW_TOKEN_BUDGET (20,000) per run
Escalation rate	memory/store/review-history.jsonl, outcome field	Track over time; a sudden spike may indicate reviewer miscalibration, not just more risky PRs
Fallback rate	orchestrator.py output: status field = escalated_fallback	Should be 0 with a valid API key configured; a nonzero rate indicates API/credential issues

2. Incident Response

Each row below maps directly to an exception type actually raised in orchestrator.py — these aren't hypothetical scenarios, they're the real failure modes the code is built to handle.

Symptom	Likely Cause	Response
status: escalated_fallback in orchestrator output	LLMUnavailableError — API key missing, invalid, or the API call failed after retries	Check ANTHROPIC_API_KEY is set and valid (echo \$ANTHROPIC_API_KEY). This is expected, safe behavior, not a bug — the pipeline correctly refused to fabricate a review. No PR was silently approved.
status: blocked_by_policy	PermissionError — an agent role attempted a tool call outside its TOOL_GRANTS	Check the audit_events in the output for which role/tool was denied. If this is unexpected (a legitimate role being blocked), compare mcp/*.py's TOOL_GRANTS against docs/governance-policy.md — they may have drifted; run tests/test_policy.py to confirm.
status: stopped_budget	BudgetExceededError — a single call or the whole run exceeded token budgets	Check budget.tokens_used in the output. If a single PR is legitimately large, consider raising PER_CALL_TOKEN_BUDGET / PER_WORKFLOW_TOKEN_BUDGET in orchestrator.py — but treat repeated hits as a signal to investigate why calls are running large, not just raise the limit.
CI policy-checks job failing	TOOL_GRANTS in a MCP server no longer matches docs/governance-policy.md	Read the specific failure message from tests/test_policy.py — it names the exact role and tool that diverged. Do not merge until code and policy doc agree. See tool-evolution-drill.md for a worked example of this exact failure.
Retrieval self-test hit rate drops below 4/6	A change to the indexing logic, relevance threshold, or underlying data regressed retrieval quality	Compare against docs/retrieval-quality-report.md's baseline. Check memory/store/calibration-proposals.jsonl for any pending threshold changes that may need human review.
MCP subprocess call times out	Server hung or crashed; PER_CALL_TIMEOUT_SECONDS (30s) exceeded	Check server stderr output. Restart is safe — servers are stateless per-call subprocesses, no persistent connection to recover.

3. Escalation Rules

These are structural rules enforced in code (`skills/risk_scoring.py`, `orchestrator.py`), not just guidance — a human should never need to remember to apply them manually.

- Any **critical** or **high** severity finding → automatically escalated, never auto-approved.
- LLM step unavailable after retries → automatically escalated (fail closed), never silently approved.
- A proposed calibration change (`memory/store/calibration-proposals.jsonl`) → requires human review and manual promotion into `calibration-log.jsonl`. No automatic promotion path exists.
- A CI policy-check failure → blocks merge. This should never be overridden without updating both the code and `docs/governance-policy.md` together, with a note in `docs/iteration-log.md` explaining why.

4. Rollback Procedures

Reverting a bad tool-grant or policy change

This exact procedure was tested end-to-end during the tool-evolution drill (`docs/tool-evolution-drill.md`):

1. Identify the incorrect change in `mcp/storage_server.py` or `mcp/retrieval_server.py`'s `TOOL_GRANTS` dict.
2. Revert the specific dict entry to match `docs/governance-policy.md`'s documented matrix.
3. Run `tests/test_policy.py` locally to confirm all 5 checks pass again.
4. Confirm functionally, not just via the test suite: make a live tool call as the affected role and verify it succeeds.
5. Log the incident and fix in `docs/iteration-log.md`.

Reverting a calibration log entry

Since `calibration-log.jsonl` is append-only and human-promoted (never written directly by an agent — see ADR-006), a bad entry is reverted by: appending a new entry documenting the reversal and its reason (never silently deleting history), then updating the affected skill or agent definition back to its prior state, and re-running the eval harness to confirm the regression is gone.

Reverting a deterministic conversion

If a deterministic conversion (e.g., `skills/risk_scoring.py`, per ADR-001) is found to be wrong: revert the code change via git, re-run the self-test (`python3 skills/risk_scoring.py`) to confirm the prior behavior is restored, and write a new ADR documenting why the conversion was reverted — do not silently delete or edit the original ADR.

5. Next Steps for Real Deployment

- Configure a real `ANTHROPIC_API_KEY` in the deployment environment's secret store (never in `.env` committed to git).
 - Move from local stdio MCP servers to the networked official MCP SDK (see ADR-004's noted open risk).
 - Add real-time monitoring/alerting on the signals in Section 1, rather than manual/on-demand checks.
 - Expand the ground-truth and holdout sets beyond the current small sample before relying on pass rates as a strong statistical claim.
 - Wire live GitHub webhooks (PR opened / ready-for-review) as the real trigger, replacing the current local-diff-file simulation used on this project's no-deployment delivery path.
-

Related documents: docs/governance-policy.md, docs/audit-log-template.md, docs/tool-evolution-drill.md, docs/reliability-cost-controls.md, docs/adr/ (ADR-001, ADR-004, ADR-006).